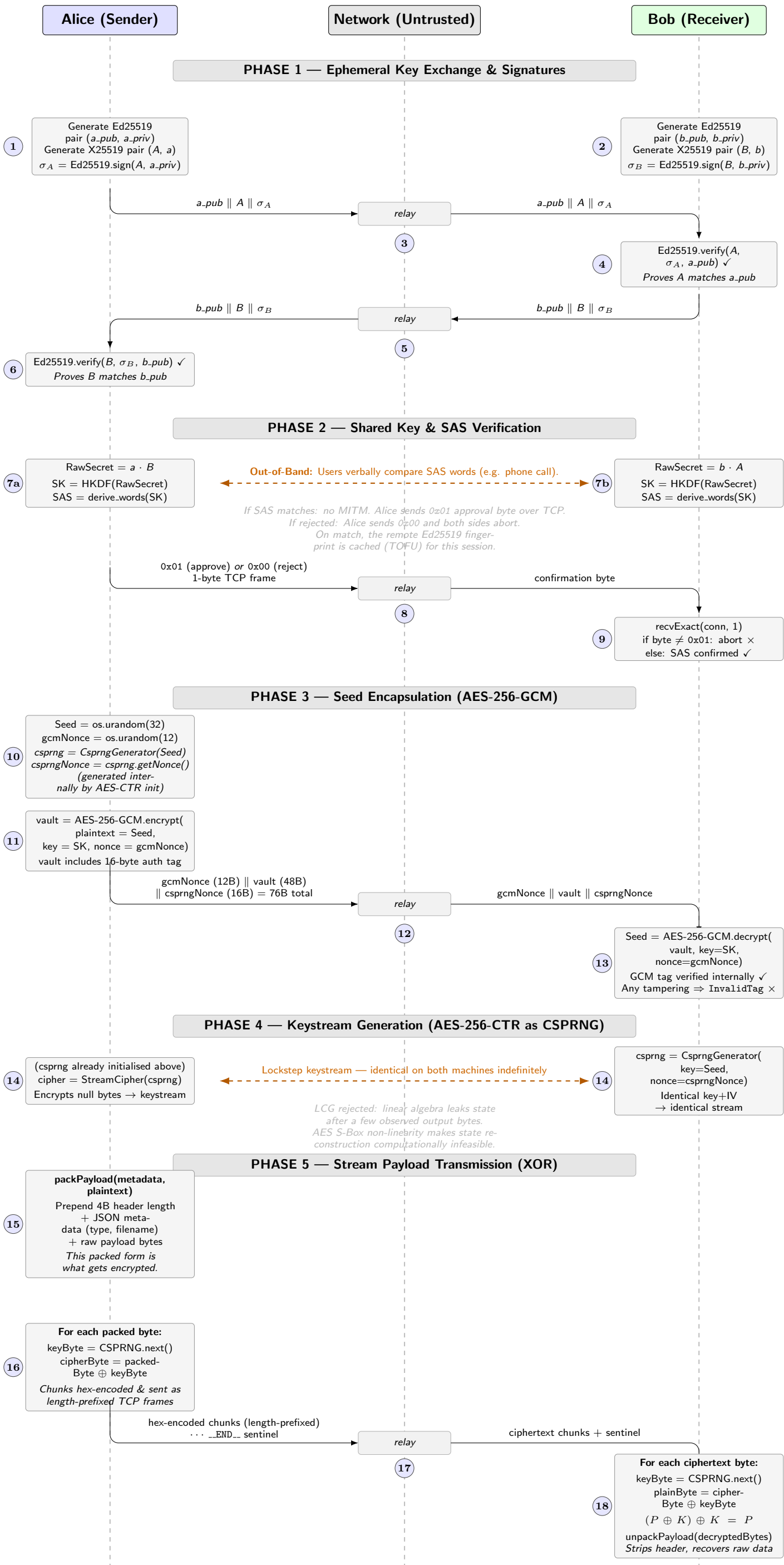


END-TO-END PROTOCOL: SECURE MESSAGING HANDSHAKE & STREAM



PROCESS EXPLANATIONS

Phase 1 — Keygen & Signatures

1-2 Ephemeral Keys: Both parties generate in-memory Ed25519 (identity) and X25519 (session) keypairs. The X25519 public keys are signed using the Ed25519 private keys. This cryptographically binds the session key exchange to the asserted identities, ensuring that an attacker cannot secretly replace the X25519 public keys without invalidating the digital signature.

3-6 Exchange & Verify: The public keys and their signatures are exchanged over the untrusted network. Each party uses the received Ed25519 public key to independently verify the signature on the X25519 key. If verification fails, a Man-in-the-Middle (MITM) attack is assumed and the connection drops immediately.

Phase 2 — Shared Key & SAS

7a-b HKDF & SAS: A Shared Key (SK) is derived independently by both parties. To mitigate "Trust-On-First-Use" vulnerabilities, a 4-word Short Authentication String (SAS) is derived from the SK via HKDF-SHA256 and displayed to each user. Users compare SAS words verbally over a secondary channel (e.g. phone call). If the words match, no MITM is present.

8-9 TCP Confirmation: Alice sends a single-byte TCP frame (0x01 = approved, 0x00 = rejected) to Bob. Bob reads this byte before proceeding. If 0x00 is received, both sides abort. On success, the remote Ed25519 fingerprint is temporarily cached (TOFU), allowing future connections in this session to silently skip the SAS step.

Phase 3 — Seed Encapsulation

10 Seed & Nonces: Alice generates a 256-bit Seed via os.urandom(32) and a 12-byte gcmNonce via os.urandom(12). She instantiates CsprngGenerator(Seed), which internally generates a 16-byte csprngNonce via os.urandom(16) during AES-CTR initialization. She retrieves it with csprng.getNonce().

11 AES-GCM Encrypt: The Seed is encapsulated using AES-256-GCM under the derived SK. GCM is an AEAD mode that natively computes a 16-byte authentication tag over the ciphertext.

12-13 Vault Transmit & Decrypt: A 76-byte "Seed Vault" is transmitted as a length-prefixed frame: gcmNonce (12B) + encrypted Seed with tag (48B) + csprngNonce (16B). Bob decrypts using AES-GCM. A single flipped bit raises an InvalidTag exception and aborts the stream.

Phase 4 — Keystream

14 AES-CTR CSPRNG: Alice's CsprngGenerator was already initialised in Phase 3. Bob instantiates his own using the received Seed and csprngNonce. Because AES is deterministic, identical key+IV produces identical keystreams indefinitely. Unlike LCG, AES S-Box non-linearity makes state reconstruction computationally infeasible.

Phase 5 — XOR Stream

15 packPayload: Before encryption, the plaintext is wrapped with a length-prefixed JSON metadata header (type, filename) via packPayload(). This packed binary blob is what the stream cipher actually encrypts, allowing the receiver to reconstruct both the payload type and raw data after decryption.

16-17 Encrypt & Transmit: Each byte of the packed payload is XORed (⊕) with the next keystream byte. Ciphertext chunks are hex-encoded, wrapped in length-prefixed TCP frames, and streamed immediately. A _END_ sentinel frame signals end-of-stream.

18 Decrypt & Unpack: Bob XORs each ciphertext byte with his identical local keystream to recover the packed payload, then calls unpackPayload() to strip the metadata header and recover the original raw bytes.

Diagram detailing the full end-to-end secure messaging protocol: Ed25519 identity, ECDH key exchange, HKDF derivation, SAS out-of-band verification with in-band TCP confirmation, AES-GCM seed encapsulation, and AES-CTR/XOR stream cipher with payload framing.